

IMAGE CAPTION GENERATOR USING CNN + LSTM

Submitted By

Name: __yarlapalli Sukadev____

Submission Date: __25-06-2026____

Abstract

This project presents an Image Caption Generator using Deep Learning by integrating Computer Vision and Natural Language Processing techniques. A pre-trained ResNet50 Convolutional Neural Network (CNN) is used to extract image features, while a Long Short-Term Memory (LSTM) network generates meaningful natural language captions.

The Flickr8K dataset is used for training and evaluation. A Streamlit web application is developed that allows users to upload an image and automatically generate a human-readable caption.

1. Introduction

Image Caption Generation is an advanced Deep Learning task that combines:

- Computer Vision
- Natural Language Processing
- Sequence Modeling

The system automatically understands image content and produces descriptive captions in natural language.

This project combines:

- ResNet50 for image feature extraction
 - LSTM for sequence generation
 - Streamlit for deployment
-

2. Objective

The objectives of this project are:

- Extract visual information from images using CNN.
- Process captions using NLP techniques.

- Train an LSTM model for caption generation.
 - Generate captions for unseen images.
 - Evaluate performance using BLEU and METEOR scores.
 - Develop an interactive web application for image captioning.
-

3. Dataset Description

Dataset:

Flickr8K Dataset

Source:

Kaggle

Number of Images:

8091

Total Captions:

40455

Captions per Image:

5

Example:

Image:

1000268201_693b08cb0e.jpg

Captions:

- A child in a pink dress is climbing up a set of stairs.
 - A girl going into a wooden building.
 - A little girl climbing into a wooden playhouse.
 - A little girl climbing the stairs to her playhouse.
 - A little girl in a pink dress going into a wooden cabin.
-

4. Data Preprocessing

The following preprocessing steps were performed:

- Convert text to lowercase.
- Remove punctuation.
- Remove single-character words.
- Remove extra spaces.
- Add special tokens:

startseq

endseq

Example:

Original:

A girl going into a wooden building.

Processed:

startseq girl going into wooden building endseq

5. Vocabulary Statistics

Vocabulary Size:

8768

Maximum Sequence Length:

34

Total Images:

8091

Total Captions:

40455

6. CNN Architecture and Feature Extraction

A pre-trained ResNet50 model was used as a feature extractor.

Configuration:

- Weights: ImageNet
- Top layer removed

- Output Feature Dimension:

(1, 2048)

Example:

Feature Vector Shape:

(1,2048)

The extracted features are used as image representations for the caption generation network.

7. Tokenizer Implementation

Tokenizer converts words into integer sequences.

Example:

Caption:

startseq little girl climbing into wooden playhouse endseq

Converted Sequence:

[1, 45, 73, 102, 20, 38, 211, 2]

Tokenizer was saved as:

tokenizer.pkl

8. Embedding Layer

Embedding Configuration:

Vocabulary Size:

8768

Embedding Dimension:

256

Mask Zero:

True

Purpose:

Transforms integer sequences into dense vector representations for LSTM processing.

9. Model Architecture

The architecture consists of two branches:

Image Branch

Input Image

↓

ResNet50

↓

Feature Vector (2048)

↓

Dense (256)

Caption Branch

Input Caption

↓

Tokenizer

↓

Embedding (8768 × 256)

↓

LSTM (256)

↓

Add Layer

↓

Dense (256)

↓

Dense (8768)

↓

Softmax

↓

Generated Caption

Insert Architecture Diagram Here

Figure 1: CNN + LSTM Image Caption Generator Architecture

10. Model Summary

Total Parameters:

5,613,632

Trainable Parameters:

5,613,632

Non-trainable Parameters:

0

11. Layer-wise Explanation

ResNet50:

Extracts deep image features.

Embedding Layer:

Converts words into dense vectors.

LSTM:

Learns sequential relationships between words.

Add Layer:

Combines image features and textual features.

Dense Layer:

Learns hidden representations.

Softmax Layer:

Predicts the next word in the caption.

12. Training Hyperparameters

Optimizer:

Adam

Loss Function:

Categorical Crossentropy

Embedding Dimension:

256

LSTM Units:

256

Batch Size:

64

Epochs:

20

Dropout:

0.5

13. Training Results

Training was performed for 20 epochs.

Initial Loss:

5.5965

Final Loss:

3.4046

The gradual reduction in training loss demonstrates successful learning and convergence of the model.

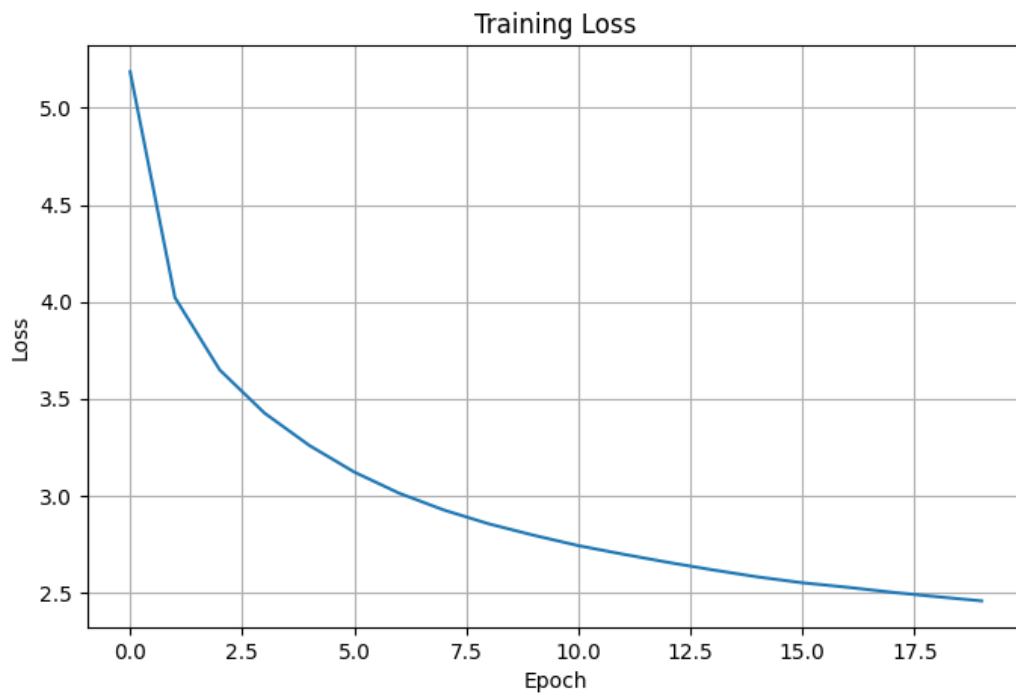


Figure 2: Training Loss Across 20 Epochs

14. Evaluation Metrics

BLEU Score

Insert BLEU Score Screenshot Here

Example:

BLEU-1: 0.49

BLEU-2:0.34

BLEU-3:0.25

BLEU-4:0.13

METEOR Score

Insert METEOR Score Screenshot Here

METEOR Score:0.34

The BLEU and METEOR scores indicate the semantic and grammatical quality of generated captions.

15. Sample Predictions

6/24/26, 5:35 PM

Image Caption Generator



Image Caption Generator

Upload an image and generate a caption using CNN + LSTM

Upload an Image



3550615...630f4f.jpg

59.1KB



Uploaded Image

Generate Caption

Caption Generated Successfully

Generated Caption

man in red shirt and blue shirt is sitting on bench with luggage

Generated Caption: man in red shirt and blue shirt is sitting on bench with luggage

16. Streamlit Web Application

A Streamlit application was developed for deployment.

Features:

- Upload image from local system
- Display uploaded image
- Extract features using ResNet50
- Generate captions using CNN + LSTM
- Display captions instantly

6/24/26, 5:35 PM

Image Caption Generator

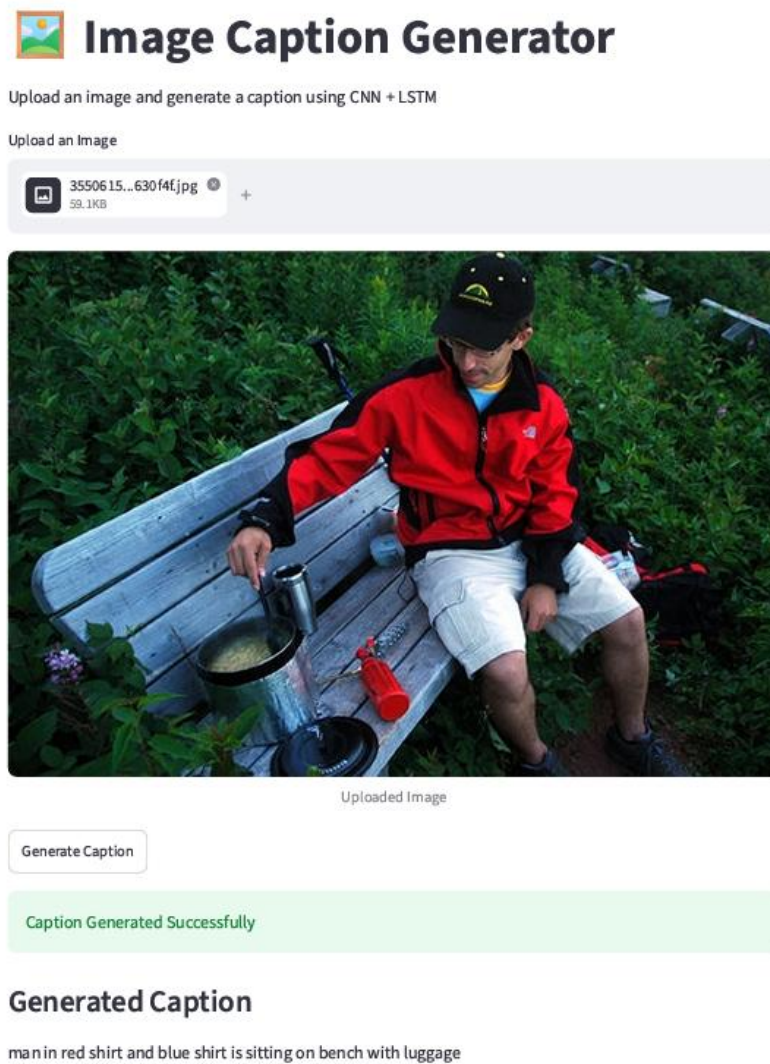


Figure 3: Streamlit User Interface

17. Deployment Instructions

Step 1

Install required packages:

```
pip install tensorflow
```

```
pip install streamlit
```

```
pip install pandas
```

```
pip install pillow
```

```
pip install nltk
```

Step 2

Run application:

```
streamlit run app.py
```

Step 3

Open browser:

<http://localhost:8501>

Step 4

Upload image and click:

Generate Caption

18. Project Structure

Image_Caption_Generator/

app.py

model.py

data_generator.py

predict.py

bleu_score.py

meteor_score.py
caption_model_trained.keras
tokenizer.pkl
history.pkl
data/
captions.txt
Images/
screenshots/
README.md
requirements.txt
Project_Report.pdf

19. Future Scope

This project can be enhanced using:

- Attention Mechanism
 - BLIP Models
 - Vision Transformers
 - Transformer Decoders
 - Larger datasets such as MS COCO
 - Multilingual Caption Generation
-

20. Conclusion

This project successfully implements an end-to-end Image Caption Generator using CNN and LSTM architectures. ResNet50 effectively extracts visual features, while the LSTM network generates meaningful human-readable captions. The developed Streamlit application provides a user-friendly interface for uploading images and generating captions instantly.

The obtained BLEU and METEOR scores demonstrate satisfactory caption generation performance and validate the effectiveness of the proposed deep learning approach.

